

ГУАП

КАФЕДРА № 14

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Старший преподаватель	14.05.2026	Н.И. Синёв
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Косвенная адресация

по курсу: Программирование на языках Ассемблера

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	2026	14.05.2026	А.Н. Корякин
1443		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Описание задачи

Написать программу с использованием косвенной адресации в ассемблере Apple ARM 64.

Формализация

Переписать из массива A в массив B те элементы массива A, индексы которых совпадают со значениями. Результат программы должен выглядеть следующим образом, результат - вывод массива A и B. Задача принимает два значения N, A, - N является размером массива A, затем вводится сам массив A, пользователем с клавиатуры. Ссылка на все исходники на GitHub: <https://github.com/Snctnm/Assembler>

Блок-схема



Рис. 1. Блок-схема программы

Исходный код программы

Код на ассемблере:

```
// Лабораторная 4. Косвенная адресация. Вариант 19.  
// A[i] == i -> копировать A[i] в B.  
// scanf/printf на macOS: variadic-аргументы передаются через стек [sp].  
  
.global _main  
  
.equ MAX_N, 32  
  
.text  
.align 2
```

```

_main:
    // Выделяем память на стеке (выровнено по 16 байт)
    sub sp, sp, #96
    stp x29, x30, [sp, #80]
    add x29, sp, #80

    // --- Ввод N ---
    sub x8, x29, #24

    mov x11, sp
    str x8, [x11]                // Кладем адрес для scanf на стек

    adrp x0, input_n_str@PAGE
    add x0, x0, input_n_str@PAGEOFF
    bl _scanf

    ldr x23, [x29, #-24]        // x23 = N

    cmp x23, #0
    b.le print_error_n

    mov x12, #MAX_N
    cmp x23, x12
    b.gt print_error_n

    adrp x20, array_a@PAGE
    add x20, x20, array_a@PAGEOFF

    mov x19, x20
    mov x25, x23                // счётчик ввода

    // Инициализируем массив B нулями
    adrp x21, array_b@PAGE
    add x21, x21, array_b@PAGEOFF
    mov x27, x23
init_b_loop:
    cbz x27, init_b_done
    str xzr, [x21], #8
    subs x27, x27, #1
    b.ne init_b_loop
init_b_done:

    // Выведем адрес переменной для отладки
    sub x8, x29, #24
    mov x11, sp
    str x8, [x11]
    adrp x0, debug_addr@PAGE
    add x0, x0, debug_addr@PAGEOFF
    bl _printf

input_loop:
    cbz x25, process_array

```

```

    sub x8, x29, #24

    mov x11, sp
    str x8, [x11]                                // Адрес для ввода элемента на стек

    adrp x0, input_elem_str@PAGE
    add x0, x0, input_elem_str@PAGEOFF
    bl _scanf

    ldr x9, [x29, #-24]

    str x9, [x19], #8

    mov x11, sp
    str x9, [x11]
    adrp x0, debug_value@PAGE
    add x0, x0, debug_value@PAGEOFF
    bl _printf

    subs x25, x25, #1
    b.ne input_loop

process_array:
    adrp x20, array_a@PAGE
    add x20, x20, array_a@PAGEOFF
    adrp x21, array_b@PAGE
    add x21, x21, array_b@PAGEOFF

    // Отладка: выводим содержимое A
    adrp x0, debug_msg@PAGE
    add x0, x0, debug_msg@PAGEOFF
    bl _printf

    mov x28, #0
debug_loop_a:
    cmp x28, x23
    b.ge debug_done

    ldr x14, [x20, x28, lsl #3]

    mov x11, sp
    str x14, [x11]
    adrp x0, debug_elem@PAGE
    add x0, x0, debug_elem@PAGEOFF
    bl _printf

    add x28, x28, #1
    b.ne debug_loop_a
debug_done:
    adrp x0, newline_str@PAGE
    add x0, x0, newline_str@PAGEOFF

```

```

bl _printf

// Основная логика: копирование из A в B, если A[i] == i
mov x10, x23
mov x24, #0 // x24 - счетчик элементов в массиве B
mov x17, #0 // x17 - индекс i

process_loop:
    cmp x17, x10
    b.ge print_result

    ldr x14, [x20, x17, lsl #3] // Загружаем A[i]

    // Условие: копировать A[i] в B если A[i] == i
    cmp x14, x17
    b.ne next_i

    str x14, [x21, x24, lsl #3] // Сохраняем в B
    add x24, x24, #1 // Увеличиваем счетчик элементов B

next_i:
    add x17, x17, #1
    b process_loop

print_result:
    mov x26, x24 // Копируем счетчик B для вывода

    adrp x0, result_msg@PAGE
    add x0, x0, result_msg@PAGEOFF
    bl _printf

    cbz x26, print_empty

    adrp x22, array_b@PAGE
    add x22, x22, array_b@PAGEOFF

print_loop:
    cbz x26, print_newline

    ldr x14, [x22], #8

    mov x11, sp
    str x14, [x11]

    adrp x0, output_elem_str@PAGE
    add x0, x0, output_elem_str@PAGEOFF
    bl _printf

    subs x26, x26, #1
    b.ne print_loop

print_newline:

```

```

    adrp x0, newline_str@PAGE
    add x0, x0, newline_str@PAGEOFF
    bl _printf
    b exit_main

print_empty:
    adrp x0, empty_msg@PAGE
    add x0, x0, empty_msg@PAGEOFF
    bl _printf
    b exit_main

print_error_n:
    adrp x0, error_msg@PAGE
    add x0, x0, error_msg@PAGEOFF
    bl _printf

exit_main:
    ldp x29, x30, [sp, #80]
    add sp, sp, #96

    mov x0, #0
    mov x16, #1 // Syscall exit (macOS)
    svc #0x80

.section __DATA, __bss
.align 3
array_a:
    .space (MAX_N * 8)
array_b:
    .space (MAX_N * 8)

.section __TEXT, __cstring, cstring_literals
input_n_str:
    .asciz "%lld"
input_elem_str:
    .asciz "%lld"
result_msg:
    .asciz "Result: "
output_elem_str:
    .asciz "%lld "
newline_str:
    .asciz "\n"
error_msg:
    .asciz "Error: N must be in 1..32\n"
empty_msg:
    .asciz "none\n"
debug_msg:
    .asciz "Array A: "
debug_elem:
    .asciz "%lld "
debug_addr:
    .asciz "Address for scanf: %p\n"

```

```
debug_value:
.asciz "Read value: %lld\n"
```

Тестирование

Выполним три сценария тестов по работе программы, с ручным вычислениям и работой программы.

Таблица 1. Копирование массива А в массив В

Набор тестовых данных	N	A	Результат(массив В)
1	3	0[0], 2[1], 2[2]	0[0], 2[2]
2	0	-	1 <= N <= 31
3	1	2[0]	none

```
snctm@MacBook-Pro-Aisen fourthLab % ./lab4
3
Address for scanf: 0x16f63eb48
0
Read value: 0
2
Read value: 2
2
Read value: 2
Array A: 0 2 2
Result: 0 2
snctm@MacBook-Pro-Aisen fourthLab % ./lab4
0
Error: N must be in 1..32
snctm@MacBook-Pro-Aisen fourthLab % ./lab4
1
Address for scanf: 0x16dafab48
2
Read value: 2
Array A: 2
Result: none
```

Рис. 2. Результат тестов в программе

Выводы

В ходе выполнения лабораторной работы была успешно решена задача разработки программы на языке ассемблера ARM64 для копирования через косвенную адресацию из массива А в массив В, все значения которых равны своим индексам.

Что было сделано:

Формализация задачи - были выполнены ввод данных N и A, реализованы механизмы отладки для нахождения ошибок и их устранения, копирование в массив В и её проверка для 0 и отрицательных значений. Разработка программы - написан код на ассемблере ARM64, включающий:

1. Выделение и освобождение стека с сохранением регистров

2. Организацию ввода двух значений через `scanf`
3. Нахождение чисел равных своим индексам
4. Копирование чисел в массив В
5. Проверка копирования
6. Отладка всей программы
7. Вывод результатов через `printf`

Тестирование - проведено ручное тестирование для трёх наборов входных данных.

Результаты работы программы полностью совпали с теоретическими расчётами. Итог: Все поставленные цели лабораторной работы достигнуты. Программа корректно работает на платформе ARM64 macOS, правильно обрабатывает ввод массива и её размера, выполняет необходимые вычисления для дальнейшего копирования и выводит результат в требуемом формате.